tidaq.

Build plan

Price history, low/high signals and checkout-trust scores for luxury hotel stays in DACH and Europe. Built by two Claude sessions; Zaid owns the business.

Version 1.0 · 11 October 2026 · Repo zaidg7/tidaq · tidaq.com

30 Nov 2026 soft launch	**< €50** per month (data + hosting)	**36–50** five-star hotels, 12 cities	**~2 h** of Zaid's time per week

OWNER	MAIN BUILDER	ON ZAID'S PC
Zaid	**CLOUD session**	**LOCAL session**
Accounts, money, decisions, checkout audit with the LOCAL session, content. A 15-minute review every Sunday. Part A.	Works the backlog top-down in the cloud. One PR per task, tests included, merges itself when CI is green.	Tasks that need a real browser or Zaid's accounts: the checkout audit with Claude in Chrome and device QA.

Start here

Part A · Your to-do list

Part C · Task backlog

Part E · Decisions

Timeline

Part B · The plan

Part D · Design system

Part F · Operating manual for Claude sessions

Not legal or tax advice. Verify legal texts before launch.

Generated from the repo docs

Start here

What happens next, and the only things you need to do this week.

Your first steps (about 2.5 hours this week)

1. **Create the empty repo and give Claude access (5 min)**
   github.com/new → zaidg7/tidaq, private, no README → add it to the Claude GitHub App. Then write "repo ready" in the chat. (Z-01)

2. **Trademark check for "Tidaq" (10 min)**
   EUIPO eSearch plus, DPMAregister, WIPO. Look at classes 35, 39, 42, 43. (Z-02)

3. **Buy tidaq.com at Cloudflare (10 min)**
   About $10/year, auto-renew on. (Z-03)

4. **Accounts and keys (45 min)**
   Cloudflare token + account ID, Supabase project in Frankfurt, DataForSEO top-up. Keys go into GitHub secrets with the exact names in Part A, never into a chat. (Z-04, Z-05)

5. **Claude environment settings (10 min)**
   Allow the listed domains and add the DataForSEO + Supabase variables, so cloud sessions can reach the services. (Z-06)

6. **Set up your PC (30 min)**
   Git, Node 22, Claude desktop app, Chrome + Claude in Chrome, clone the repo. (Z-07)

7. **Start the CLOUD session**
   New claude.ai Code session with the tidaq repo, paste the CLOUD start prompt (Part B §14). It works the backlog by itself from there.

How the machine runs

1 · Pick	2 · Claim	3 · Build	4 · Prove	5 · Ship
first open task in TASKS.md, priority order	draft PR titled [ID], so no double work	code + tests, small commits	CI green, checklist, self-review	squash-merge, auto-deploy, next task

Blocked on you? The session writes exact steps into docs/BLOCKERS.md and moves on. Every Sunday you get a GitHub issue "Weekly summary" with what shipped, what's next and what you need to do.

On the Claude Pro plan: run one session at a time (CLOUD by default; LOCAL only for its tasks or the audit). If milestones slip by more than a week, upgrade to Max 5× and run both in parallel.

Timeline

Targets, not promises. Data collection is the critical path: the earlier the collector runs, the better the charts at launch. ★ = milestone.

Week	Dates	Phase	Zaid	Sessions (task IDs)	Outcome
W42	12–18 Oct	Setup	Z-01 to Z-07, Z-10 (~2.5 h)	C-001, C-002, C-003, C-007, C-004, C-005, C-010	Repo, CI, accounts, spike report
W43	19–25 Oct	Data	Sunday review	C-012, C-014, C-015, C-011, C-013a	Catalog of 36–50 hotels, collector posting
W44	26 Oct – 1 Nov	Data	Sunday review	C-013b, C-006, C-020a	Data flowing daily ★
W45	2–8 Nov	MVP	Audit sessions start	C-016, C-032, C-020b, C-021	Admin + audit tool, homepage
W46	9–15 Nov	MVP	Audit 2–3×	C-022, C-023, C-026, C-027	Stay page, hotel calendar, legal
W47	16–22 Nov	MVP	Audit 2–3×, Z-09	C-024, C-025, C-028, C-029	Feature-complete on staging ★
W48	23–29 Nov	Alerts	Audit, Z-12 legal read	C-030, C-031, C-034, C-041, L-040	Alerts live, QA, 150 tests
W49	30 Nov – 6 Dec	Launch	Z-13 videos, posting	C-042, C-043, C-044	tidaq.com live ★
Dec–Mar		Growth	Z-11 affiliate apps, content	C-050 to C-056	First €100 commission by March

Goals after launch

When	Goal
Dec 2026	Trust report + first video series; 500 visitors
Mar 2027	Affiliate programs approved; first €100 in commission
Jun 2027	5,000 visitors/month, 1,000 active price watches; then decide on Pro tier or B2B reports

PART A

Your to-do list

Only what the sessions can't do for you, with exact steps.

Only things the sessions can't do for you: your identity, money, logins, decisions and your face on camera. Everything else is outsourced.

Golden rule: never paste a key, token or password into a chat, issue or PR. Put keys only where a step says (GitHub secrets, Cloudflare, the Claude environment settings).

Total for week 1: **about 2.5 hours.** After that: about 1–3 hours a week, mostly the checkout audit and content.

Week 1 (Mon 12 – Sun 18 Oct 2026)

Z-01 · Create the GitHub repo and give Claude access (5 min). Do this first, everything waits for it.

1. Open github.com/new
2. Owner `zaidg7`, name `tidaq`, **Private**. Leave "Add a README" **unticked** (the repo must be empty).
3. Create repository.
4. Open github.com/settings/installations → **Claude** → Configure → Repository access → add `tidaq` → Save. (If Claude isn't listed: install it from github.com/apps/claude and choose `tidaq`.)
5. Repo → Settings → General → Pull Requests: tick **Allow squash merging** and **Automatically delete head branches**.
6. Write "repo ready" in the chat. Claude pushes the plan into it.

Z-02 · Trademark check for "Tidaq" (10 min)

1. EUIPO: euipo.europa.eu → eSearch plus → search `TIDAQ` (also try `TIDAK`, `TIDAX`).
2. DPMA: register.dpma.de → DPMAregister → Marken → Einsteigerrecherche → `Tidaq`.
3. WIPO: branddb.wipo.int → search `TIDAQ`.
4. What matters: an identical or very similar name in classes **35, 39, 42 or 43** (advertising/comparison, travel, software, hotels). If you find one, write it in the chat before buying the domain. If nothing comes up, continue.

- Registering your own EU trademark (about €850) can wait until there's revenue.

Z-03 · Buy tidaq.com (10 min, ~$10/year)

1. Create a Cloudflare account at dash.cloudflare.com (use a strong password + 2FA).
2. Domain Registration → Register Domains → `tidaq.com` → buy (1 year, auto-renew on).
3. Done. DNS stays in Cloudflare; the sessions configure it through the deploy pipeline.

Z-04 · Accounts and keys (45 min)

Cloudflare (already created in Z-03)

1. Workers & Pages → open it once so Workers is activated (free plan for now).
2. My Profile → API Tokens → Create Token → template **Edit Cloudflare Workers** → All accounts / your account, zone `tidaq.com` → Create. Copy the token.
3. Copy your **Account ID** (Workers & Pages overview, right sidebar).
4. Later, only if a session asks in BLOCKERS: Workers Paid plan ($5/month).

Supabase

1. supabase.com → New project → name `tidaq`, region **Frankfurt (eu-central-1)**, free plan. Save the database password in your password manager. (If the cloud session already created it through the Supabase connector, skip this.)
2. Project Settings → API: copy **Project URL**, **anon/publishable key** and **service_role/secret key**.

DataForSEO

1. dataforseo.com → sign up, verify email.
2. Add funds: the smallest top-up offered (around $50 lasts about 2 months at our budget).
3. Dashboard → API Access: copy the **API login** and **API password** (different from your account password).

Put the keys into GitHub (repo `tidaq` → Settings → Secrets and variables → Actions → New repository secret). One secret per line, exact names:

Secret name	Value
`CLOUDFLARE_API_TOKEN`	the Cloudflare token
`CLOUDFLARE_ACCOUNT_ID`	your Account ID
`SUPABASE_URL`	Project URL
`SUPABASE_ANON_KEY`	anon/publishable key
`SUPABASE_SERVICE_ROLE_KEY`	service_role/secret key
`DATAFORSEO_LOGIN`	API login
`DATAFORSEO_PASSWORD`	API password

Z-05 · Resend for emails (10 min)

1. resend.com → sign up → Domains → Add domain `tidaq.com` → follow the Cloudflare DNS steps (Resend can add the records automatically if you connect Cloudflare).
2. API Keys → Create → "Sending access" → copy.
3. Add GitHub secret `RESEND_API_KEY`.

Z-06 · Claude cloud environment settings (10 min)

In a claude.ai Code session for `tidaq`: environment menu in the session title bar → **Edit**.

1. **Network access:** keep "Limited" and add these **Allowed domains** (keep "Allow package managers" ticked): `api.dataforseo.com`, `docs.dataforseo.com`, `*.supabase.co`, `api.supabase.com`, `api.cloudflare.com`, `*.workers.dev`, `tidaq.com`, `www.tidaq.com`, `api.resend.com`, `developers.cloudflare.com`, `opennext.js.org`, `nextjs.org`
2. **Network secrets / environment variables** (whichever the settings page offers): `DATAFORSEO_LOGIN`, `DATAFORSEO_PASSWORD`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`. Same values as in GitHub.

3. Save. New sessions pick it up.

Z-07 · Set up your PC for the LOCAL session (30 min, once)

1. Install **Git for Windows** (git-scm.com) with default options.
2. Install **Node.js 22 LTS** (nodejs.org). Then open PowerShell and run `corepack enable`.
3. Install the **Claude desktop app** (claude.ai/download) and sign in. Claude Code is inside it.
4. Install **Google Chrome** and the **Claude in Chrome** extension (Chrome Web Store), sign in.
5. Clone the repo: in PowerShell `cd $HOME\Documents` then `git clone https://github.com/zaidg7/tidaq.git`.
6. Open the `tidaq` folder in Claude Code and paste the LOCAL start prompt (PLAN §14). The LOCAL session can help with anything that fails.

Z-08 · Every Sunday: 15-minute review

Open the newest **Weekly summary** issue on GitHub (you'll get a notification), do the listed steps, close it.

Weeks 2–7 (19 Oct – 29 Nov)

Z-07b · Checkout audit sessions (2–3× a week, 30–45 min each). Your most important job before launch.

With the LOCAL session (task L-033): Claude in Chrome does the clicking and reading; you watch, approve sites when asked and make sure nothing gets booked. Goal: 150 tests by 29 Nov. **Never enter payment or personal data.** Film a few of these sessions (screen recording + your voice). That's your launch content.

Z-09 · Google Search Console (5 min, when a session asks)

search.google.com/search-console → add property `tidaq.com` (Domain) → verify via Cloudflare DNS (Search Console offers a Cloudflare button).

Z-10 · Reserve social handles (10 min, this week if possible)

TikTok, Instagram, YouTube, X: `@tidaq` (or `@tidaq.de` / `@tidaqhotels` if taken). Use the same profile picture (the session makes one from the logo).

Z-12 · Read the legal pages once (20 min, week of 23 Nov)

Read `/impressum`, `/datenschutz`, `/affiliate-hinweis`, `/ranking` on staging. Check your name, address and email. If you have a Steuernummer, add it as the GitHub issue comment the session asks for (it's public on the Impressum anyway).

Z-13 · Content (your part, from launch week)

The sessions write scripts and make the charts. You record and post. Start with 3 videos in launch week:

1. "Ich habe 150 Hotel-Checkouts getestet. Das ist passiert."
2. "Dieses 5-Sterne-Hotel ist gerade 14 % günstiger als sonst" (live chart)
3. "Booking vs. Hotel direkt: wer ist wirklich günstiger?"

After launch (Dec 2026 onward)

Z–11 · Affiliate programs (60 min total, spread out; after the site is live with content)

The session prepares the application texts and exact links in a BLOCKERS entry. You sign up because it needs your identity and tax details:

- Awin publisher account → apply to **Booking.com** (or CJ if Booking assigns that)
- **Expedia Group** affiliate program (Expedia, Hotels.com)
- **Trip.com** and **Agoda** affiliate programs Then put each affiliate ID where the session tells you (usually a GitHub secret).

Z–14 · Finances (ongoing)

Keep invoices from Cloudflare, DataForSEO, Resend and later affiliate payouts for your Kleinunternehmer bookkeeping. Commissions are business income.

Your weekly time budget

Week	What	Time
12–18 Oct	Z-01 to Z-07, Z-10	~2.5 h
19 Oct – 29 Nov	audit sessions, Sunday review, Z-09, Z-12	~1.5–2.5 h/week
Launch week (30 Nov)	Z-13 videos, posting	~3 h
Dec onward	Sunday review, content, Z-11	~2–3 h/week

PART B

The plan

Product, architecture, data model, methods, legal, risks.

Version 1.0 · 11 Oct 2026 · Owner: Zaid Ghazal · Builders: Claude CLOUD + LOCAL sessions

This is the master plan. `docs/TASKS.md` is the work queue, `docs/DECISIONS.md` the locked decisions, `docs/DESIGN.md` the design system, `docs/ZAID.md` Zaid's own to-do list.

---

1. What Tidaq is

One sentence: Tidaq shows you how the price of *your* luxury hotel stay is moving, whether right now is a low or a high, and which booking site actually charges at checkout what it shows.

Example: You want the Hotel Adlon in Berlin, 1–3 Dec 2026, 2 adults, with breakfast. Tidaq shows:

- today's best price per provider (Booking.com, Expedia, the hotel's own website …)
- a chart of how that price moved since we started tracking it, with the 30-day average and the lowest point
- a clear signal: **Nahe Tiefpunkt / Günstig / Normal / Teuer**, plus the trend (falling, flat, rising)
- a **trust score per provider**: how often its shown price matched the real checkout total in our tests
- a button: **Preis beobachten**. You get an email when the price drops or hits a low.

Why now / the gap:

- Google Hotels shows a little price info but no clear history, no breakfast or room filters, and its cheapest offers often come from providers that charge more at checkout.
- Google penalizes partners whose checkout price doesn't match (its Price Accuracy Policy), which proves mismatches are common. Nobody shows users *which* providers are reliable.
- In Germany hotels may sell cheaper on their own website (the BGH struck down Booking.com's best-price clause), so real price gaps between channels exist.
- On a €1,500 stay, a 10 % move is €150. Luxury guests notice that, and our commission per booking is high.

---

2. Users

Primary: the aspirational luxury traveller. 25–45, DACH, books a five-star hotel 1–3× a year for a city break, anniversary or New Year's Eve. Price-aware, researches for weeks, hates overpaying, doesn't use luxury travel agents.

Jobs to be done:

1. "Tell me if I should book now or wait."
2. "Show me the real price, not a teaser that grows at checkout."
3. "Watch it for me and tell me when it drops."
4. "Show me luxury deals in cities I like."

Not our user (v1): business travel managers, budget travellers, the ultra-rich who book through Virtuoso agents or Amex FHR.

3. Scope

v1 (launch target Mon 30 Nov 2026)

- ~50 five-star hotels in 12 cities (D-04), tracked daily on a rolling grid of stay dates
- Stay page with chart, stats, signal, advice, provider table with trust score
- Hotel page with a **price calendar** (cheapest upcoming dates)
- City pages and a deals page
- Email price alerts with double opt-in
- Trust score from manual checkout tests (≥ 150 tests by launch)
- Methodology, ranking transparency, affiliate disclosure, Impressum, Datenschutz
- German + English, mobile-first, Soft Modern design

Not in v1 (on purpose)

- User accounts with passwords (email-only watches instead)
- Any hotel in the world (only our catalog; "request a hotel" form instead)
- Room-type and breakfast filters, **unless** the C-010 spike shows the API returns that data reliably
- Paid plans, "watch my booking" rebooking, B2B reports
- Native apps

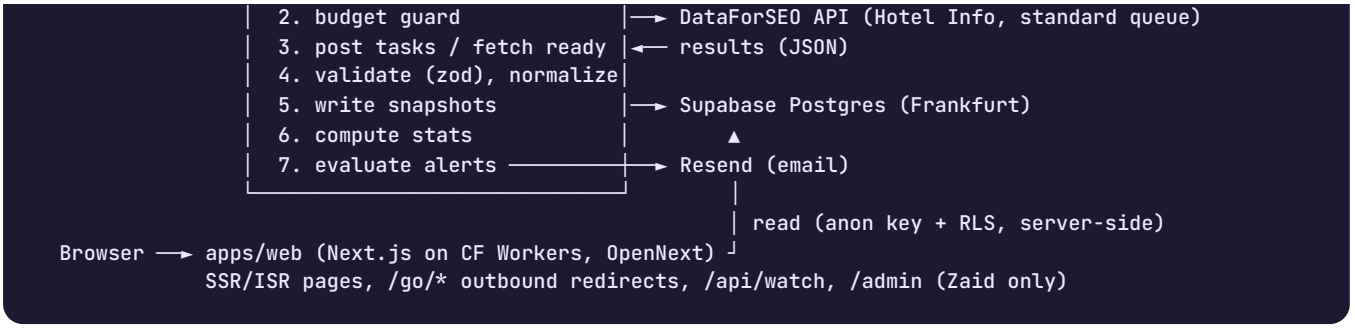
4. Goals and milestones

When	Milestone	Measure
Sun 18 Oct 2026	Setup done	Repo, CI, domain, accounts, spike report merged
Sun 1 Nov 2026	**Data flowing**	Collector runs daily for ≥ 36 hotels, cost within cap
Sun 22 Nov 2026	MVP feature-complete	Home, stay, hotel, city, deals, legal pages live on staging
Mon 30 Nov 2026	**Soft launch**	tidaq.com live, alerts working, ≥ 150 checkout tests published
Dec 2026	First content wave	Trust report + TikTok series; 500 visitors
Mar 2027	First money	Affiliate programs approved; **first €100 in commission**
Jun 2027	Traction	5,000 visitors/month, 1,000 active watches

The earlier the collector runs, the better the charts at launch. **Data collection is the critical path.** Everything else can be polished later.

5. Architecture





- **apps/web:** Next.js App Router, React Server Components, ISR for hotel/city pages (revalidate after each collector run via on-demand revalidation or a short revalidate time). All data reads are server-side.
- **apps/collector:** a plain Cloudflare Worker with `scheduled()` handler and Cron Triggers. Posts DataForSEO tasks, then fetches results with `tasks_ready` polling in a second cron run or a postback URL (`/postback` endpoint on the worker, protected by a secret token in the URL). Prefer postback if the spike confirms it works; otherwise poll.
- **packages/core:** pure TypeScript logic shared by both: zod schemas, normalization, price stats, trust score, budget guard. 100 % unit-testable without network.
- **Supabase:** Postgres + Storage (audit screenshots) + Auth (only for the admin and, optionally, magic links). Row Level Security on every table; the web app reads public views with the anon key; writes go through server routes with the service role.
- **CI/CD:** GitHub Actions. PR: lint, typecheck, unit, build, Playwright smoke. Main: same + deploy web and collector with Wrangler + apply Supabase migrations.

6. Data model (v1)

Source of truth: `supabase/migrations/`. This is the starting design; C-004 implements it.

```
-- catalog
create table cities (
  id smallserial primary key,
  slug text unique not null,            -- 'berlin'
  name_de text not null, name_en text not null,
  country_code char(2) not null
);

create table hotels (
  id serial primary key,
  slug text unique not null,            -- 'hotel-adlon-kempinski-berlin'
  name text not null,
  city_id smallint not null references cities(id),
  stars smallint not null default 5,
  dfs_hotel_identifier text unique,     -- DataForSEO / Google hotel id
  lat double precision, lng double precision,
  official_url text,
  image_url text, image_credit text,    -- licensed images only (see PLAN §11)
  active boolean not null default true,
  created_at timestamptz not null default now()
);

create table providers (
  id serial primary key,
  slug text unique not null,            -- 'booking-com', 'hotel-direct'
  name text not null,
  domain text,
  is_direct boolean not null default false,
```

```
  affiliate_template text              -- URL template with {url}, filled when approved
);
create table provider_aliases (        -- maps raw API source names to providers
  alias text primary key,
  provider_id int not null references providers(id)
);

-- what we track
create table tracked_stays (
  id bigserial primary key,
  hotel_id int not null references hotels(id),
  check_in date not null,
  check_out date not null,
  adults smallint not null default 2,
  origin text not null check (origin in ('grid','watch','manual')),
  active boolean not null default true,
  created_at timestamptz not null default now(),
  unique (hotel_id, check_in, check_out, adults),
  check (check_out > check_in and check_out - check_in ⩽ 30)
);

-- raw observations (append-only)
create table price_snapshots (
  id bigserial primary key,
  stay_id bigint not null references tracked_stays(id),
  provider_id int not null references providers(id),
  captured_at timestamptz not null,
  total_cents int not null,            -- total for the whole stay, EUR, incl. taxes as shown
  currency char(3) not null default 'EUR',
  room_name text, board text,          -- if the API returns them
  free_cancellation boolean,
  raw jsonb not null                   -- the original offer item for re-processing
);
create index on price_snapshots (stay_id, captured_at desc);

-- one row per stay per day: the best price and per-provider best (derived, rebuilt by the collector)
create table stay_daily (
  stay_id bigint not null references tracked_stays(id),
  day date not null,
  best_cents int not null,
  best_provider_id int references providers(id),
  per_provider jsonb not null,         -- {provider_id: cents}
  primary key (stay_id, day)
);

-- computed signal per stay (latest)
create table stay_stats (
  stay_id bigint primary key references tracked_stays(id),
  updated_at timestamptz not null,
  days_tracked int not null,
  now_cents int, low_cents int, high_cents int, avg30_cents int,
  percentile numeric,                  -- 0..1, position of today's price in the window
  trend_7d numeric,                    -- % change per week from regression
  state text check (state in ('collecting','near_low','good','normal','high'))
);

-- trust: manual checkout tests
create table trust_tests (
  id bigserial primary key,
  provider_id int not null references providers(id),
  hotel_id int not null references hotels(id),
  check_in date not null, check_out date not null, adults smallint not null,
  listed_cents int not null,           -- price shown on Google Hotels for this provider
  checkout_cents int,                  -- final total on the last page before payment (null = could not
book)
  outcome text not null check (outcome in ('match','higher','lower','unavailable')),
```

```
  extras text,                          -- e.g. 'city tax added', 'breakfast not included'
  screenshot_path text,                 -- Supabase Storage, private bucket
  tested_at timestamptz not null default now(),
  tested_by text not null default 'zaid'
);
create table provider_scores (           -- derived
  provider_id int primary key references providers(id),
  updated_at timestamptz not null,
  tests int not null, matches int not null,
  avg_markup_pct numeric, score smallint not null
);

-- users and alerts (no accounts, email-only)
create table watchers (
  id uuid primary key default gen_random_uuid(),
  email text unique not null,
  locale text not null default 'de',
  confirmed_at timestamptz,             -- double opt-in
  consent_text text not null, consent_at timestamptz not null,
  unsubscribed_at timestamptz,
  created_at timestamptz not null default now()
);
create table watches (
  id uuid primary key default gen_random_uuid(),
  watcher_id uuid not null references watchers(id) on delete cascade,
  stay_id bigint not null references tracked_stays(id),
  target_cents int,                     -- optional "tell me below X"
  last_alert_cents int, last_alert_at timestamptz,
  active boolean not null default true,
  created_at timestamptz not null default now(),
  unique (watcher_id, stay_id)
);
create table alerts_sent (
  id bigserial primary key,
  watch_id uuid not null references watches(id) on delete cascade,
  kind text not null,                   -- 'target','new_low','drop','near_low'
  price_cents int not null,
  sent_at timestamptz not null default now(),
  resend_id text
);

-- ops
create table collector_runs (
  id bigserial primary key,
  started_at timestamptz not null, finished_at timestamptz,
  tasks_posted int not null default 0, tasks_done int not null default 0,
  est_cost_usd numeric not null default 0,
  errors jsonb not null default '[]'
);
create table events (                    -- privacy-friendly product analytics, no IP, no user id
  id bigserial primary key,
  name text not null,                   -- 'search','stay_view','watch_created','outbound_click'
  props jsonb not null default '{}',
  at timestamptz not null default now()
);
```

RLS: public `select` only on catalog tables, `stay_daily`, `stay_stats`, `provider_scores` and on public views. Everything with emails (`watchers`, `watches`, `alerts_sent`) has **no** public access. Server routes use the service role.

7. Data collection

7.1 The stay grid

Charts only exist for stays we track *before* someone asks. Each day the collector makes sure every active hotel has these stays tracked:

- **Weekends:** the next 8 Fri→Sun stays (2 nights)
- **Midweek:** 4 Tue→Thu stays spread over the next 8 weeks
- **Events:** New Year's Eve (30 Dec→1 Jan), Christmas markets (first two December weekends), Easter weekend, city specials (Oktoberfest, Art Basel …) in a `special_dates` config
- **Plus** every stay a user watches (origin `watch`)

All for 2 adults (the most common luxury booking). About 14 grid stays per hotel.

7.2 Budget math (verify in C-010)

- Hotel Info, standard queue: ~$0.0008 per task (one hotel entity per task).
- 50 hotels × 14 stays = 700 tasks/day. Daily run: $0.56/day ≈ **$17/month**.
- A second evening run only for stays checking in within 21 days (prices move faster then): ~150 tasks/day ≈ $3.6/month.
- User watches: capped by `DAILY_TASK_CAP`.
- **Total data ≈ $20–25/month**, plus Cloudflare Workers paid $5, domain ~$1/month. Inside €50.
- DataForSEO needs a prepaid balance. The first top-up lasts 2+ months.

Budget guard (C-013): before posting, count today's tasks in `collector_runs`. If adding the batch would exceed `DAILY_TASK_CAP` or the month-to-date estimate would exceed `MONTHLY_BUDGET_USD`, post only the highest-priority tasks (watches first, then stays closest to check-in, then the rest) and log the skip. Unit-tested.

7.3 Normalization

- Request `currency=EUR`, `language_code=de`, location Germany.
- Store the **total for the stay** as shown by the provider, in cents. If the API returns a nightly price, multiply by nights and mark it in `raw`.
- Map raw source names to `providers` via `provider_aliases`. Unknown sources are auto-created as providers with `slug` from the domain and logged for review.
- Drop offers with price ≤ 0, wrong currency or missing source. Keep everything else in `raw` for re-processing.

8. Methods

8.1 Price signal (per stay)

Input: `stay_daily.best_cents` for the last N days (N = days tracked, max 60).

- **Collecting:** fewer than 7 days of data. Show "Wir sammeln noch Daten (x/7 Tage)" and the chart so far.
- **Percentile:** where today's best price sits in the window (0 = lowest ever seen, 1 = highest).
- **State:** `near_low` if today ≤ low × 1.02 or percentile ≤ 0.10; `good` if percentile ≤ 0.35; `high` if percentile ≥ 0.75; else `normal`.

- **Trend:** slope of a least-squares fit over the last 7 days, as % per week. Falling < −1.5 %, rising > +1.5 %, else flat.
- **Advice text (DE examples):**
  - near_low: "Jetzt buchen ist sinnvoll. Der Preis liegt nahe am Tiefpunkt der letzten {n} Tage."
  - good + falling: "Guter Preis, und er fällt noch. Beobachten lohnt sich."
  - high: "Gerade teuer. Wir sagen dir Bescheid, wenn er fällt."
  - Within 7 days of check-in, add: "Kurz vor Anreise steigen Preise oft. Warte nicht zu lange."
- Never promise the future. Wording: "lohnt sich wahrscheinlich", never "wird fallen".

8.2 Trust score (per provider)

From `trust_tests` (manual checkout tests, PLAN §9 and task L-033):

1. For each test: markup = (checkout − listed) / listed. A test **passes** if markup ≤ +1 % (same price or cheaper). `unavailable` (the price could not be booked) fails.
2. accuracy = passed tests / all tests (n).
3. penalty = mean of the positive markups of the failed tests × 200, capped at 40 points (e.g. an average +14 % markup gives 28 points).
4. raw_score = clamp(100 × accuracy − penalty × (1 − accuracy), 0, 100).
5. **Bayesian shrinkage** for small samples: score = round((n × raw_score + 5 × 70) / (n + 5)), so a provider with 2 perfect tests shows 79, not 100.
6. Recompute after every new test. Unit-test with fixed examples.

- Show the score **with the sample**: "96 · 24 von 25 Tests stimmten". Under 5 tests: show "Noch zu wenig Tests" instead of a number.
- Methodology page explains all of this in plain German.

8.3 Ranking in the provider table

Sort by **trusted price**: listed price adjusted by the provider's average markup (if score is known), so a teaser price from an unreliable provider doesn't sit on top. Show the listed price and, when relevant, "erwartet an der Kasse ≈ 1.212 €". **Commission never affects ranking.** This is stated on the ranking transparency page (§ 5b UWG requires disclosing the main ranking parameters).

---

9. Pages and features (v1)

All pages in DE (`/…`) and EN (`/en/…`). Mobile-first. Soft Modern design.

Route (DE)	What it does	Key acceptance criteria
`/`	Hero with search, live example stay, how it works, trust teaser, deals, CTA	Matches reference design; live example uses real data; LCP < 2.5 s on 4G
`/hotel/[slug]`	Hotel overview: photo, city, **price calendar** (heatmap of best price per tracked stay, next 60 days), best current deals, providers	Calendar works on mobile (horizontal scroll by month), tap → stay page

Route (DE)	What it does	Key acceptance criteria
`/hotel/[slug]/[checkin]--[checkout]?adults=2`	**Stay page**: price now, state chip, delta vs avg30, 60-day chart, low/avg/high, advice, provider table with trust score + "erwartet an der Kasse", Preis beobachten	Chart accessible (table fallback), empty/collecting states, untracked stay → create on demand within cap and show "Erste Preise in ca. 1 Stunde"
`/stadt/[city]`	Luxury hotels in a city with current state per hotel	SEO title "Luxushotels in Berlin: Preisverlauf & Deals"
`/deals`	Biggest drops vs 30-day average in the last 7 days, filter by city	Only stays with ≥ 14 days of data
`/anbieter/[slug]`	Provider trust page: score, tests, typical extras	Facts only, with test dates
`/methodik`	How price signal and trust score work	Plain language, examples
`/ranking`	Ranking transparency (§ 5b UWG)	States that commission doesn't affect ranking
`/watch/confirm`, `/watch/manage/[token]`	Double opt-in confirmation; manage or unsubscribe all watches	One-click unsubscribe works without login
`/go/[offerId]`	Outbound redirect to the provider with affiliate params if available; logs an `outbound_click` event	`rel="sponsored nofollow"` on links; 302; no personal data logged
`/impressum`, `/datenschutz`, `/affiliate-hinweis`	Legal	See §11
`/admin/*`	Zaid only: data health, audit entry, hotel catalog, budget	Supabase Auth magic link, email must be in `ADMIN_EMAILS`

Search (header and hero): autocomplete over our hotels and cities (client-side list, under 100 items). Date range picker with German locale, nights shown, max 30 nights, no past dates. Adults 1–4. Toggles (breakfast, refundable) shown only if C-010 proves the data exists.

Watch flow: stay page → "Preis beobachten" → email field + optional target price + consent checkbox (not pre-ticked) → confirmation email → click → active. Each alert email has the price, the chart image (OG image endpoint), the best provider, and one-click unsubscribe. Max one email per watch per day.

Alert triggers: price ≤ target; new low (lower than every previous best price, and at least 2 % below the last alert); drop ≥ 5 % since the last alert; state changes to `near_low`.

OG images: dynamic per stay and hotel (price, chart sparkline, state chip). Great for sharing and TikTok screenshots.

10. SEO and content

- Programmatic but useful pages: hotel pages, city pages, "Beste Zeit, das [Hotel] zu buchen" (after 30+ days of data), "Silvester in [Stadt]: Luxushotel-Preise".
- Structured data: `Hotel` with `address`, `starRating`, `priceRange`. Don't mark up offers we don't sell.
- hreflang de/en, sitemap per locale, canonical URLs, fast pages.
- Content for Zaid (TikTok/Instagram/Reddit/mydealz), drafted weekly by a session from fresh data: "Dieses Hotel ist gerade 14 % günstiger", "Wir haben 150 Checkouts getestet", "Booking vs. Hotel direkt".

11. Legal and compliance (Germany)

Not legal advice. Before launch, Zaid reads the final texts once (task Z-12).

- **Impressum (§ 5 DDG):** Zaid Ghazal, address and email as on AheadSignal, Kleinunternehmer § 19 UStG, Steuernummer when available. Cite DDG, not TMG.
- **Datenschutzerklärung (DSGVO):** processors Cloudflare (hosting, cookieless analytics), Supabase (EU, database), Resend (email); purposes; legal bases (Art. 6(1)(a) consent for alerts, (f) legitimate interest for logs); retention; rights; no tracking cookies. Sign DPAs/AVVs (Supabase, Resend and Cloudflare offer them online).
- **Email alerts:** double opt-in (§ 7 UWG); store consent text + timestamp; unsubscribe link in every email; send only what was asked for.
- **Affiliate disclosure:** visible note near outbound links ("Wir erhalten ggf. eine Provision. Für dich ändert sich der Preis nicht.") and a full `/affiliate-hinweis` page. Links `rel="sponsored nofollow"`.
- **Ranking transparency (§ 5b UWG, Omnibus directive):** page describing main ranking parameters and that commissions don't influence ranking.
- **Prices (PAngV):** show totals including taxes as the provider shows them; label the stay length; say when city tax may be extra.
- **Comparisons and provider claims (UWG §§ 4, 6):** factual, verifiable, dated. Keep screenshots of every test. Neutral wording.
- **Images:** only licensed images (hotel press kits with permission, Wikimedia Commons with attribution, or our own illustrations). No Google or Booking photos.
- **Trademarks:** check "Tidaq" in EUIPO/DPMA before buying ads (Z-02). Hotel and provider names are used descriptively only, no logos without permission.
- **Data source terms:** DataForSEO provides the data under its terms; we don't scrape providers ourselves.

12. Monetization

- v1: affiliate links. Programs to join once the site is live with content (Z-11): Booking.com (via Awin or CJ, around 4 % per completed stay reported by third parties), Expedia Group affiliate program (Expedia, Hotels.com), Trip.com, Agoda. Commission is paid after the stay, so the first money arrives weeks after the first bookings.
- Until approved, outbound links are plain links. That's fine; we collect click data to show the networks.
- Later: Pro tier (longer history, unlimited watches, rebooking alerts), "watch my booking" success fee, B2B rate-parity reports for hotels (2027+).

13. Risks

Risk	Impact	Mitigation
DataForSEO data lacks per-provider detail or breakfast/room info	Weaker product	C-010 spike first; fall back to "best price per provider" only; SerpApi as backup source
Costs higher than estimated	Budget	Budget guard, daily cap, cost dashboard, weekly cost in summary
Few visitors at launch	No revenue	Content-led launch from the checkout audit; SEO pages from real data; mydealz/Reddit

Risk	Impact	Mitigation
Affiliate programs reject a young site	No revenue	Apply after launch with content; several networks; direct links still give value
Legal complaint from a provider	Stress, cost	Facts only, screenshots, neutral wording, quick correction process
Pro plan usage limits slow the build	Delays	Small tasks, one session at a time, upgrade to Max 5× if milestones slip > 1 week
Zaid has little time	Blocked tasks	All Zaid tasks batched, with exact steps, in `docs/ZAID.md`
Supabase free project pauses	Downtime	Daily collector keeps it active; upgrade if needed
Next.js on Workers limits (bundle size, CPU)	Deploy problems	Workers paid plan; keep pages server-light; ISR

14. Operating model

- **CLOUD session:** main builder. Works the backlog top-down, one PR per task, merges on green CI.
- **LOCAL session:** on Zaid's PC. Takes LOCAL tasks (checkout audit with Claude in Chrome, Windows setup, real-device QA, account dashboards with Zaid present).
- **Zaid:** ZAID tasks only (accounts, payments, identity, trademark, content, final legal read), ~1–3 h/week. A 15-minute review every Sunday: read the weekly summary issue, clear blockers.
- **On Pro:** one session at a time, CLOUD by default. On Max 5×: CLOUD and LOCAL in parallel, in different areas (LOCAL never edits the same files as an open CLOUD PR).

Start prompts (copy-paste)

CLOUD session (claude.ai → Code → new session with repo `zaidg7/tidaq`):

> You are the Tidaq CLOUD build session. Read CLAUDE.md and follow its session protocol exactly. Work through docs/TASKS.md top-down: take the next TODO task owned by CLOUD whose dependencies are DONE and that has no open PR. One PR per task, tests included, merge yourself when CI is green. Continue with the next task until you run low on usage or only blocked tasks remain. Log anything you need from me in docs/BLOCKERS.md with exact steps. Don't ask me questions in chat unless something is truly impossible without me.

LOCAL session (Claude Code on the PC, in the cloned `tidaq` folder):

> You are the Tidaq LOCAL session on my Windows 11 PC. Read CLAUDE.md and follow its session protocol. Take the next TODO task owned by LOCAL. If none is open, take a CLOUD task marked parallel-safe that has no open PR. Never make bookings or enter payment or personal data on any website. Log anything you need from me in docs/BLOCKERS.md.

Weekly check (Zaid, Sundays): open the newest "Weekly summary" issue on GitHub, do the listed Zaid steps, close the issue.

PART C

Task backlog

The work queue for the CLOUD and LOCAL sessions, with acceptance criteria.

The work queue. Sessions take the **first** task that is `TODO`, has their owner, has all dependencies `DONE` and has no open PR (see CLAUDE.md §2). Zaid's own tasks (`Z-xx`) are explained step by step in `docs/ZAID.md`; they appear here only as dependencies.

Status values: `TODO` · `IN PROGRESS` (open PR exists) · `BLOCKED` (see BLOCKERS.md) · `DONE` **Owners:** `CLOUD` (cloud session) · `LOCAL` (session on Zaid's PC) · `ZAID` (human) **Sizes:** S ≈ under 1 hour of session work · M ≈ 1–3 h · L ≈ 3–6 h (split L tasks into a/b when possible) **parallel-safe:** LOCAL may take it when no LOCAL task is open.

Overview

Priority order = this table, top to bottom. Data collection is the critical path (charts need history), so data tasks come before UI polish, and the trust audit tool comes early so the checkout tests can start three weeks before launch.

ID	Title	Owner	Size	Depends on	Status
Phase 0: Setup (week 42, target Sun 18 Oct)					
C-001	Monorepo scaffold (Next.js + OpenNext + collector + core)	CLOUD	M	Z-01	TODO
C-002	CI pipeline (lint, typecheck, test, build, e2e smoke)	CLOUD	M	C-001	TODO
C-003	Session tooling (SessionStart hook, permissions allowlist)	CLOUD	S	C-001	TODO
C-007	i18n setup (de default, en) and formatting helpers	CLOUD	S	C-001	TODO
C-004	Supabase schema v1, RLS, seed providers and cities	CLOUD	M	C-001, Z-04	TODO
C-005	Deploy pipeline to Cloudflare (web + collector), staging and prod	CLOUD	M	C-002, Z-03, Z-04	TODO
Phase 1: Data (weeks 42–44, target Sun 1 Nov: data flowing)					
C-010	DataForSEO spike: real response shape, cost, feasibility report	CLOUD	M	C-001, Z-04, Z-06	TODO
C-012	Stay grid generator	CLOUD	S	C-004	TODO
C-014	Normalization and provider mapping	CLOUD	M	C-010	TODO
C-015	Price stats engine (state, percentile, trend, advice)	CLOUD	M	C-001	TODO

ID	Title	Owner	Size	Depends on	Status
C-011	Hotel catalog: resolve seed hotels, identifiers, images	CLOUD	M	C-004, C-010	TODO
C-013	Collector worker: cron, posting, results, budget guard	CLOUD	L	C-005, C-010, C-011, C-012, C-014	TODO
Phase 2: MVP website (weeks 44–47, target Sun 22 Nov)					
C-006	Design tokens and base UI components (Soft Modern)	CLOUD	L	C-001	TODO
C-020a	App shell: layout, navbar, footer, locale switch, admin auth	CLOUD	M	C-006, C-007	TODO
C-016	Admin: data health and budget dashboard	CLOUD	M	C-013, C-020a	TODO
C-032	Trust audit tool and provider scores (early, so the audit starts in week 45)	CLOUD	M	C-020a, C-004	TODO
L-033	Checkout audit with Claude in Chrome (150 tests, recurring until launch)	LOCAL	L (recurring)	C-032, Z-07	TODO
C-020b	Homepage (reference design, real live example)	CLOUD	L	C-020a, C-015	TODO
C-021	Search: autocomplete, date range picker, guests	CLOUD	M	C-020a, C-011	TODO
C-022	Stay page with chart, stats, advice, provider table	CLOUD	L	C-020a, C-015, C-013	TODO
C-023	Hotel page with price calendar	CLOUD	M	C-022	TODO
C-026	Outbound link service `/go/*` with click logging	CLOUD	S	C-022	TODO
C-027	Legal and transparency pages	CLOUD	M	C-020a	TODO
C-024	City pages	CLOUD	S	C-023	TODO
C-025	Deals page	CLOUD	S	C-022	TODO
C-028	SEO foundation and dynamic OG images	CLOUD	M	C-022	TODO
C-029	Analytics events (cookieless)	CLOUD	S	C-020a	TODO
Phase 3: Alerts and trust (week 48, target Sun 29 Nov)					
C-030	Watch flow with double opt-in and manage/unsubscribe	CLOUD	M	C-022, Z-05	TODO
C-031	Alert engine and emails	CLOUD	M	C-030, C-013	TODO

ID	Title	Owner	Size	Depends on	Status
C-034	Provider pages and methodology page with live data	CLOUD	S	C-032	TODO
Phase 4: Launch (week 49, launch Mon 30 Nov)					
L-040	Real-device QA on PC and iPhone, fix list	LOCAL	M	C-031, C-034	TODO
C-041	Launch hardening: perf, a11y, error monitoring, backups	CLOUD	M	C-031	TODO
C-042	Trust report page "150 Checkouts getestet"	CLOUD	S	L-033, C-034	TODO
C-043	Launch content pack for Zaid	CLOUD	S	C-042	TODO
C-044	Go live on tidaq.com	CLOUD	S	C-041, L-040, Z-12	TODO
Phase 5: Growth (Dec 2026 – Mar 2027)					
C-050	Affiliate wiring per approved program	CLOUD	S	Z-11	TODO
C-051	Weekly content pack routine	CLOUD	S	C-044	TODO
C-052	"Beste Buchungszeit" and event SEO pages	CLOUD	M	C-044 (30+ days data)	TODO
C-053	Coverage expansion and "request a hotel"	CLOUD	M	C-044	TODO
C-054	Weekly deals newsletter (opt-in)	CLOUD	M	C-031	TODO
C-055	Breakfast/room filters (if the spike allows)	CLOUD	M	C-010 result	TODO
C-056	Research: "watch my booking" rebooking alerts (v2)	CLOUD	S	C-044	TODO

Zaid tasks (sessions update this table when they see a Z-task is done)

ID	What	Status
Z-01	GitHub repo + Claude app access	DONE
Z-02	Trademark check	TODO
Z-03	Buy tidaq.com (Cloudflare)	TODO
Z-04	Cloudflare, Supabase, DataForSEO accounts + GitHub secrets	TODO
Z-05	Resend + `RESEND_API_KEY`	TODO
Z-06	Claude cloud environment: allowed domains + env vars	TODO
Z-07	PC setup for LOCAL session + Claude in Chrome	TODO
Z-09	Google Search Console	TODO
Z-10	Social handles	TODO

ID	What	Status
Z-11	Affiliate program applications (after launch)	TODO
Z-12	Read legal pages before launch	TODO

Tasks marked **parallel-safe** for LOCAL (no shared files with typical CLOUD work): C-003, C-007, C-015, C-027, C-029.

Phase 0: Setup

C-001 · Monorepo scaffold

Owner: CLOUD · Size: M · Depends on: Z-01 · Status: TODO · PR: –

Goal: a working pnpm monorepo with the target structure from CLAUDE.md §4.

Steps

1. pnpm workspace with `apps/web`, `apps/collector`, `packages/core`. Node 22 (`.nvmrc`, `engines`).
2. `apps/web`: latest stable Next.js 15 (App Router, TypeScript strict, `src/`-less layout as in CLAUDE.md), Tailwind CSS, ESLint (next + typescript-eslint), Prettier. Add `@opennextjs/cloudflare` and `wrangler.jsonc` following the official OpenNext Cloudflare guide; `pnpm preview` must run the worker locally.
3. `apps/collector`: Cloudflare Worker (TypeScript) with `fetch` (health + postback) and `scheduled` handlers, its own `wrangler.jsonc` with two cron triggers (`0 5 * * *`, `0 17 * * *`).
4. `packages/core`: TypeScript library, Vitest configured, exports `formatPrice`, `nights` and the zod setup.
5. Root scripts: `lint`, `typecheck`, `test`, `build`, `e2e`, `format`.
6. `.gitignore` (node_modules, .next, .open-next, .wrangler, .env*, coverage), `.editorconfig`, `.nvmrc`.
7. Replace the placeholder README section "Development" with real commands.

Done when

- [ ] `pnpm install && pnpm lint && pnpm typecheck && pnpm test && pnpm build` pass from a clean clone
- [ ] `pnpm --filter web preview` serves a placeholder page that says "Tidaq kommt bald"
- [ ] `pnpm --filter collector dev` runs and `/health` returns `{ ok: true }`

C-002 · CI pipeline

Owner: CLOUD · Size: M · Depends on: C-001 · Status: TODO · PR: –

Goal: every PR is checked automatically in under ~6 minutes.

Steps

1. `.github/workflows/ci.yml` on `pull_request` and `push` to `main`: checkout, pnpm with cache, install, lint, typecheck, unit tests with coverage, build web (OpenNext build) and collector, Playwright smoke (`apps/web/e2e/smoke.spec.ts`: home loads in de and en, has no console errors).
2. Use Playwright's Chromium. Upload the Playwright report as an artifact on failure.
3. `concurrency` group per branch to cancel outdated runs.
4. Add a coverage threshold for `packages/core` (90 % lines), failing CI below it.

Done when

- [ ] A PR shows one required-looking check "CI" that runs all steps
- [ ] An intentionally failing test makes CI red (prove it in the PR, then revert)
- [ ] Typical run under 6 minutes

C-003 · Session tooling

Owner: CLOUD · Size: S · Depends on: C-001 · Status: TODO · PR: –

Goal: new sessions start fast and ask fewer permission questions.

Steps

1. `.claude/settings.json` with an allowlist for read-only and routine commands (pnpm scripts, git status/diff/log, playwright test).
2. A SessionStart hook that runs `pnpm install --frozen-lockfile` when `node_modules` is missing (use the `session-start-hook` skill if available).
3. Add a "Quick start for sessions" section to README.

Done when

- [ ] A fresh cloud session can run `pnpm test` without manual install

C-004 · Supabase schema v1

Owner: CLOUD · Size: M · Depends on: C-001, Z-04 · Status: TODO · PR: –

Goal: the database from PLAN §6, as migrations.

Steps

1. `supabase/` folder with the CLI config. Migration `0001_init.sql` with all tables from PLAN §6, indexes, constraints.
2. RLS on every table. Public read only on `cities`, `hotels`, `providers`, `stay_daily`, `stay_stats`, `provider_scores` and on views `v_stay_public`, `v_hotel_calendar`. No public access to `watchers`, `watches`, `alerts_sent`, `trust_tests`, `collector_runs`, `events`.
3. Seed migration: the 12 cities (D-04) and core providers (Booking.com, Expedia, Hotels.com, Trip.com, Agoda, HRS, Hotel direkt / official site, Check24) with aliases.
4. Generate TypeScript types into `packages/core/src/db.types.ts`.
5. Apply to the Supabase project via the Supabase MCP tools (`apply_migration`) or the CLI in CI. Document which in README.
6. Write SQL tests for RLS (pgTAP or a Vitest script that uses the anon key against a local Supabase) or at minimum a documented manual check in the PR.

Done when

- [ ] Migrations apply cleanly on a fresh database
- [ ] With the anon key, selecting from `watchers` returns nothing / is denied
- [ ] Types are generated and used by `packages/core`

C-005 · Deploy pipeline

Owner: CLOUD · Size: M · Depends on: C-002, Z-03, Z-04 · Status: TODO · PR: –

Goal: merging to `main` deploys automatically. Two environments.

Steps

1. Staging: `tidaq-web-staging` and `tidaq-collector-staging` on workers.dev, deployed on every push to `main`. Production: `tidaq.com`, deployed when a GitHub release/tag `v*` is created (manual promote) **or** on every main push after launch (decide in the PR and update DECISIONS).
2. Wrangler deploy steps in a `deploy.yml` workflow using `CLOUDFLARE_API_TOKEN` and `CLOUDFLARE_ACCOUNT_ID`. Push Worker secrets from GitHub secrets with `wrangler secret put` (only if changed) or `wrangler secret bulk`.
3. Staging uses `ALERTS_DRY_RUN=true` and a lower `DAILY_TASK_CAP` (50).
4. Supabase: one project for now. Staging and prod share the database but staging writes to nothing user-facing. Collector staging runs only on manual trigger (`workflow_dispatch`).
5. Custom domain `tidaq.com` and `www` redirect on the prod web worker (DNS in Cloudflare).

Done when

- [ ] Merge to main → staging URL updated within ~5 minutes
- [ ] Prod deploy procedure documented in README and tested once (placeholder page on tidaq.com)

C-006 · Design tokens and base components

Owner: CLOUD · Size: L (split a/b) · Depends on: C-001 · Status: TODO · PR: –

Goal: the Soft Modern design system from `docs/DESIGN.md` as code.

Steps

1. Tokens as CSS variables + Tailwind theme (colors, radii, shadows, fonts, spacing scale). Plus Jakarta Sans via `next/font/google`.
2. Components in `apps/web/components/ui/`: Button (primary, ghost, sizes), Input, Field (label + hint + error), Chip/Toggle, Card, Badge, StateChip (collecting/near_low/good/normal/high), PriceDelta, TrustScore (score + sample text, "Noch zu wenig Tests"), Price (formatting via `formatPrice`), Sparkline, Skeleton, EmptyState, Logo (wordmark `tidaq`).
3. `/dev/ui` page (noindex, only outside production) showing every component in every state.
4. Visual regression: Playwright screenshot test of `/dev/ui` at 390 and 1280 px.

Done when

- [ ] `/dev/ui` matches DESIGN.md (colors, type, radius) side by side with the reference HTML
- [ ] Keyboard focus visible on all interactive components; contrast AA

C-007 · i18n and formatting

Owner: CLOUD · Size: S · Depends on: C-001 · Status: TODO · PR: –

Steps

1. next-intl with locales `de` (default, no prefix) and `en` (`/en`). Middleware, `messages/de.json`, `messages/en.json`.

2. `packages/core/format.ts`: `formatPrice(cents, locale)` → `1.184 €` / `€1,184`; `formatDateRange` → `1.–3. Dez. 2026` / `1–3 Dec 2026`; `formatPercent`. Unit tests for both locales, including thousands, 0, negative deltas.
3. Locale switch component (keeps the current path).

Done when

- [ ] `/` German, `/en` English; all strings from message files
- [ ] Formatting tests pass

Phase 1: Data

C-010 · DataForSEO spike (critical)

Owner: CLOUD · Size: M · Depends on: C-001, Z-04, Z-06 · Status: TODO · PR: –

Goal: know exactly what the API gives us before building on it. **Write findings, not assumptions.**

Steps

1. Script `scripts/spike/dataforseo.ts` (runs locally or via a `workflow_dispatch` GitHub Action with secrets) that:
   - calls Hotel Searches for "Luxushotel Berlin" (location Germany, language de, EUR) and records hotel identifiers,
   - calls Hotel Info for 3 hotels (Adlon, Ritz-Carlton Berlin, Waldorf Astoria Berlin) for 2 stays (next Fri–Sun, and 1–3 Dec 2026), adults 2, currency EUR,
   - tries `load_prices_by_dates` / `prices_start_date` / `prices_end_date` if available (a price calendar would save cost),
   - tests both task_post + tasks_ready + task_get and the postback mechanism (if postback is possible from the worker).
2. Save responses (they contain no personal data) to `tests/fixtures/dataforseo/`.
3. Write `docs/research/dataforseo.md` answering:
   - Which fields exist per offer: provider/source name, URL, price, nightly vs total, taxes, currency, room name, board/breakfast, free cancellation, official-site flag?
   - How many providers per hotel on average? Is "official site" included?
   - Real cost per call (account balance before/after), latency, failure rate.
   - Does the price calendar feature work, and what does it cost?
   - **Recommendation:** grid size, run times, whether breakfast/room filters are feasible (C-055), whether a second source is needed.
4. If anything contradicts PLAN §6–§8, propose changes in the same PR (update PLAN and add a DECISIONS entry).

Done when

- [ ] Report merged with real numbers
- [ ] Fixtures committed, zod schemas drafted in `packages/core/src/dataforseo.ts`
- [ ] Cost of the spike under $1

C-011 · Hotel catalog

Owner: CLOUD · Size: M · Depends on: C-004, C-010 · Status: TODO · PR: –

Goal: about 36–50 five-star hotels in the catalog with correct identifiers.

Seed list (verify each on Google Hotels via the API; replace any that are closed, rebranded or not five-star):

- Berlin: Hotel Adlon Kempinski · The Ritz-Carlton Berlin · Waldorf Astoria Berlin · Hotel de Rome
- München: Mandarin Oriental Munich · Hotel Bayerischer Hof · Rosewood Munich · The Charles Hotel
- Hamburg: The Fontenay · Fairmont Hotel Vier Jahreszeiten · Park Hyatt Hamburg · The Westin Hamburg (Elbphilharmonie)
- Frankfurt: Villa Kennedy · Steigenberger Icon Frankfurter Hof · Sofitel Frankfurt Opera
- Wien: Hotel Sacher Wien · Park Hyatt Vienna · Rosewood Vienna · Hotel Imperial
- Zürich: Baur au Lac · The Dolder Grand · Park Hyatt Zurich
- Paris: Le Bristol · Shangri-La Paris · Four Seasons Hotel George V
- London: The Savoy · Claridge's · The Ritz London
- Mailand: Four Seasons Hotel Milano · Park Hyatt Milano · Mandarin Oriental Milan
- Rom: Hotel Hassler Roma · Hotel de Russie · Hotel Eden
- Barcelona: Hotel Arts Barcelona · Mandarin Oriental Barcelona · Majestic Hotel & Spa
- Amsterdam: Waldorf Astoria Amsterdam · Conservatorium Hotel · Hotel De L'Europe

Steps

1. Script `scripts/catalog/resolve.ts`: for each seed hotel, search via the API, pick the match (name + city), store `dfs_hotel_identifier`, coordinates, official URL. Print uncertain matches for manual review in the PR description.
2. Images: only licensed. Use Wikimedia Commons images with license + attribution (store `image_credit`), else no photo and a designed gradient placeholder (DESIGN.md). Never Google/Booking photos.
3. Slugs: `hotel-adlon-kempinski-berlin` style, ASCII, stable.
4. Seed migration or idempotent upsert script; document how to add a hotel.

Done when

- [ ] ≥ 36 hotels active with identifiers, all 12 cities covered
- [ ] Uncertain matches listed in the PR and resolved

C-012 · Stay grid generator

Owner: CLOUD · Size: S · Depends on: C-004 · Status: TODO · PR: –

Goal: `packages/core/grid.ts` produces the stay list from PLAN §7.1 for a given date.

Steps

1. Pure function `buildGrid(today, hotels, specialDates)` → stays (hotel_id, check_in, check_out, adults=2).
2. Rules: next 8 Fri→Sun; 4 Tue→Thu over the next 8 weeks; special dates from `packages/core/specialDates.ts` (New Year's Eve, first two December weekends, Easter, Oktoberfest weekends for Munich, etc.); no past dates; dedupe.
3. Collector uses it daily and upserts `tracked_stays` (origin `grid`). Stays whose check-in has passed get `active=false`.
4. Unit tests with fixed "today" values (e.g. a Friday, the last days of December, a leap year).

Done when

- [ ] Tests cover weekday edge cases and year boundaries

C-013 · Collector worker

Owner: CLOUD · Size: L (split: a = posting + guard, b = results + storage) · Depends on: C-005, C-010, C-011, C-012, C-014 · Status: TODO · PR: –

Goal: prices flow into the database every day, within budget. **The most important task of the project.**

Steps

1. Morning run (05:00 UTC): refresh grid → build job list (all active stays; watches first) → budget guard (PLAN §7.2, unit-tested in core) → post DataForSEO Hotel Info tasks in batches (max 100 per request) with a `tag` = stay_id → record in `collector_runs`.
2. Results: postback endpoint `/postback?token=…` (secret), or a poll of `tasks_ready` every run plus a retry run (decide from C-010). Validate with zod → normalize (C-014) → insert `price_snapshots` → rebuild `stay_daily` for touched stays → recompute `stay_stats` (C-015) → trigger alert evaluation (C-031, once it exists).
3. Evening run (17:00 UTC): only stays with check-in within 21 days, plus watches.
4. Idempotency: re-running the same day must not double-insert (unique key on stay, provider, day, or dedupe by task id).
5. Failure handling: retries with backoff for 5xx; log errors in `collector_runs.errors`; if a run fails or the cost estimate exceeds 80 % of the monthly budget, email Zaid via Resend (`ADMIN_EMAILS`).
6. Manual trigger: `workflow_dispatch` action or an authenticated `/run` endpoint for staging.
7. Integration test: run the full pipeline against fixtures with a mocked fetch and an in-memory/local database.

Done when

- [ ] On staging, one manual run fills `price_snapshots` for ≥ 3 hotels
- [ ] Production cron enabled; after 2 days `stay_daily` has 2 rows per active stay
- [ ] Budget guard proven by a test where the cap cuts the job list in the right priority order
- [ ] Zaid gets an email when a run fails (tested by forcing an error on staging)

C-014 · Normalization and provider mapping

Owner: CLOUD · Size: M · Depends on: C-010 · Status: TODO · PR: –

Steps

1. `packages/core/normalize.ts`: raw offer → `{providerKey, totalCents, roomName?, board?, freeCancellation?, raw}` per PLAN §7.3.
2. Alias table handling; unknown providers create a pending provider and log once.
3. Detect nightly vs total price; detect and drop obvious outliers (price < 30 % or > 400 % of the stay's median that day, logged, not stored as best).
4. Unit tests from fixtures, including German number strings if the API returns them ("1.184,00 €").

Done when

- [ ] All fixture offers normalize without errors; outlier rule tested

C-015 · Price stats engine

Owner: CLOUD · Size: M · Depends on: C-001 · Status: TODO · PR: –

Steps

1. `packages/core/stats.ts`: from a series of daily best prices → low, high, avg30, percentile, trend_7d (least squares), state, advice key (PLAN §8.1). Pure functions.
2. Advice copy keys in next-intl (`advice.near_low`, …) with DE and EN text; the 7-days-before-check-in note.
3. Tests: flat series, falling, rising, single point, 6 points (collecting), spike outlier, near-low boundary.

Done when

- [ ] ≥ 95 % coverage on stats.ts; outputs match PLAN §8.1 examples

C-016 · Admin: data health and budget

Owner: CLOUD · Size: M · Depends on: C-013, C-020a · Status: TODO · PR: –

Steps

1. `/admin` (auth from C-020a): last 14 collector runs (tasks, cost, errors), month-to-date cost vs budget, coverage (active hotels, stays, % stays with today's data), unknown providers to map, hotels with no data in 3 days.
2. Buttons: "Run collector now (staging)", "Deactivate hotel".

Done when

- [ ] Zaid can see on his phone in 10 seconds whether data is healthy

---

Phase 2: MVP website

C-020a · App shell and admin auth

Owner: CLOUD · Size: M · Depends on: C-006, C-007 · Status: TODO · PR: –

Steps

1. Root layout per DESIGN.md: navbar (logo, links Preisverlauf · Vertrauens-Check · Deals · So funktioniert's, CTA), footer (legal links, affiliate note, locale switch, © 2026 Tidaq).
2. 404 and error pages in brand style.
3. Admin auth: Supabase magic link to an email in `ADMIN_EMAILS`; middleware protects `/admin/*`.
4. `robots` noindex on staging.

Done when

- [ ] Shell works at 390 px and 1280 px; Lighthouse a11y ≥ 95

C-020b · Homepage

Owner: CLOUD · Size: L · Depends on: C-020a, C-015 · Status: TODO · PR: –

Steps

1. Build the homepage section by section like `docs/design/reference-homepage.html` (Soft Modern variant): hero + search card, live example panel, how it works (3 steps), trust teaser with figures, deals of the week, final CTA, footer.
2. The live example uses a real featured stay (config: the stay with the most data and an interesting state); falls back to a static example labeled "Beispiel" if data is missing.
3. Copy from the reference, refined; DE and EN.

Done when

- [ ] Side-by-side with the reference: same structure and feel; real data visible
- [ ] Mobile LCP < 2.5 s (Lighthouse mobile ≥ 90)

C-021 · Search

Owner: CLOUD · Size: M · Depends on: C-020a, C-011 · Status: TODO · PR: –

Steps

1. Autocomplete for hotels and cities (accessible combobox pattern, diacritics-insensitive: "zurich" finds "Zürich").
2. Date range picker: German locale, Monday first, nights count, max 30 nights, no past dates, mobile bottom sheet.
3. Guests: adults 1–4 (children later). Toggles only if C-055 is done.
4. Submit → stay page URL; city → city page.

Done when

- [ ] Works with keyboard only and on iPhone width; e2e test covers a full search

C-022 · Stay page

Owner: CLOUD · Size: L (split: a = data + layout + chart, b = provider table + watch CTA + states) · Depends on: C-020a, C-015, C-013 · Status: TODO · PR: –

Steps

1. Route `/hotel/[slug]/[checkin]--[checkout]?adults=2` (validate params; redirect invalid).
2. Header: hotel, dates, guests, state chip, price now (big), delta vs avg30, nights and total.
3. Chart component (SVG): best price per day, avg30 dashed line, low marker, today marker, hover/tap tooltip with date and price, y-axis in €, accessible data table toggle.
4. Stats row: Tief / Schnitt / Hoch. Advice box.
5. Provider table: per provider the latest price, trust score (or "Noch zu wenig Tests"), "erwartet an der Kasse ≈ …" if markup is known, outbound button via `/go/*`, sorted by trusted price (PLAN §8.3). Affiliate note under the table.
6. States: collecting (< 7 days), no data yet (untracked stay → create `tracked_stays` origin `watch` if within cap → "Erste Preise in ca. 1 Stunde"), hotel inactive, stay in the past.
7. "Preis beobachten" CTA (opens the C-030 flow; until then it's disabled with "Bald verfügbar").

Done when

- [ ] Every state has a designed screen; e2e tests for tracked, collecting and untracked stays

C-023 · Hotel page with price calendar

Owner: CLOUD · Size: M · Depends on: C-022 · Status: TODO · PR: –

Steps

1. `/hotel/[slug]`: photo/placeholder, name, city, short facts, price calendar: for each tracked check-in date in the next 60 days, a cell colored by price level (relative to this hotel's range), tap → stay page. Legend. Month navigation on mobile.
2. Cheapest upcoming weekend and midweek stays highlighted.
3. Hotel structured data.

Done when

- [ ] Calendar readable at 390 px; color is not the only signal (price text in cells)

C-024 · City pages

Owner: CLOUD · Size: S · Depends on: C-023 · Status: TODO · PR: – `/stadt/[city]`: hotel cards with photo, cheapest upcoming stay, state chip. SEO title and intro text (DE/EN). Links from footer and home.

C-025 · Deals page

Owner: CLOUD · Size: S · Depends on: C-022 · Status: TODO · PR: – `/deals`: stays with ≥ 14 days of data whose best price is ≥ 8 % below avg30, sorted by % drop; city filter chips; each card links to the stay page. Empty state if no deals.

C-026 · Outbound link service

Owner: CLOUD · Size: S · Depends on: C-022 · Status: TODO · PR: – `/go/[offerId]`: look up the latest offer URL, apply `providers.affiliate_template` if set, log `outbound_click` (provider, hotel, stay; no IP, no user agent stored), 302 redirect. Links carry `rel="sponsored nofollow noopener"`, open in a new tab. Unit tests for template filling and unknown ids.

C-027 · Legal and transparency pages

Owner: CLOUD · Size: M · Depends on: C-020a · Status: TODO · PR: –

Steps

1. `/impressum`: Zaid Ghazal, Prinz-Heinrich-Straße 17, 24106 Kiel, Deutschland; E-Mail zaidghazal216@gmail.com (until a @tidaq.com address exists); "Gemäß § 19 UStG wird keine Umsatzsteuer berechnet (Kleinunternehmerregelung)"; Steuernummer line as placeholder `[STEUERNUMMER]` and a BLOCKERS entry for Zaid. Heading "Angaben gemäß § 5 DDG".
2. `/datenschutz`: per PLAN §11 (Cloudflare, Supabase EU, Resend, cookieless analytics, alert emails with double opt-in, rights, contact). Plain German, structured headings.
3. `/affiliate-hinweis`, `/ranking` (§ 5b UWG), `/methodik` (static version; C-034 adds live numbers).
4. English versions (`/en/imprint`, `/en/privacy`, …) noting that the German version is binding.
5. BLOCKERS entry for Zaid to read all legal texts once before launch (Z-12).

Done when

- [ ] All pages linked in the footer; no TMG references anywhere

C-028 · SEO foundation and OG images

Owner: CLOUD · Size: M · Depends on: C-022 · Status: TODO · PR: – Metadata per page (DE/EN), hreflang, canonical, sitemap index per locale, robots (staging noindex), JSON-LD (Organization, Hotel), dynamic OG images for stay and hotel pages (price, sparkline, state chip, brand) via `next/og` (check Workers compatibility; fall back to a pre-rendered image route). Google Search Console verification via DNS (Z-09).

C-029 · Analytics events

Owner: CLOUD · Size: S · Depends on: C-020a · Status: TODO · PR: – Cloudflare Web Analytics snippet (cookieless) + server-side `events` inserts for search, stay_view, watch_created, watch_confirmed, outbound_click. No cookies, no IPs, no user ids. A small weekly stats query for the summary.

Phase 3: Alerts and trust

C-030 · Watch flow

Owner: CLOUD · Size: M · Depends on: C-022, Z-05 · Status: TODO · PR: –

Steps

1. Form on the stay page: email, optional target price, consent checkbox (not pre-ticked, text stored), Cloudflare Turnstile against bots.
2. Server route creates/updates `watchers` + `watches`; sends a confirmation email (Resend, React Email template DE/EN) with a signed token link.
3. `/watch/confirm?token=` activates; `/watch/manage/[token]` lists watches, delete one, unsubscribe all (one click, no login).
4. Rate limit per email and IP (in memory/KV, IP not stored).
5. Dev and staging use `ALERTS_DRY_RUN=true` (emails logged, not sent).

Done when

- [ ] e2e test of the whole flow with the dry-run mailer
- [ ] Unconfirmed watches never receive alerts; unconfirmed watchers are deleted after 7 days

C-031 · Alert engine and emails

Owner: CLOUD · Size: M · Depends on: C-030, C-013 · Status: TODO · PR: – After each collector run: evaluate active, confirmed watches against triggers (PLAN §9), throttle to 1 email/watch/day, send via Resend with the price, state, mini chart (OG image URL), best trusted provider and the manage/unsubscribe link (List-Unsubscribe header). Record in `alerts_sent`. Unit tests for every trigger and the throttle.

C-032 · Trust audit tool

Owner: CLOUD · Size: M · Depends on: C-020a, C-004 (scheduled early in Phase 2) · Status: TODO · PR: –

Steps

1. `/admin/audit`: a fast mobile-friendly form: pick hotel + stay (prefilled from tracked stays), provider, listed price (as shown on Google Hotels), checkout total, outcome, extras (chips: Citytax extra, Frühstück fehlt, anderes Zimmer, Gebühren, nicht buchbar), screenshot upload (private Storage bucket), notes.

2. List of recent tests with edit/delete.
3. Score computation (PLAN §8.2) in `packages/core/trust.ts`, unit-tested, written to `provider_scores` after each change.
4. CSV export.

Done when

- [ ] Entering one test takes under 60 seconds on a phone

L-033 · Checkout audit with Claude in Chrome (recurring)

Owner: LOCAL · Size: L, recurring (10–15 tests per session) · Depends on: C-032, Z-07 · Status: TODO · PR: –

Goal: 150+ real checkout tests by launch, across ≥ 8 providers and ≥ 30 hotels. This is the trust score's data and the launch story.

Protocol per test (with Zaid at the PC, or Zaid approving each site in Claude in Chrome):

1. Pick a tracked stay (rotate cities, providers, dates; prefer the stays shown on Google Hotels with 4+ providers).
2. Open Google Hotels for that hotel and dates, 2 adults. Note each provider's listed price.
3. For one provider: click through to the provider site, select the same room type as the cheapest listed, continue to the **last page before payment**. **Never enter personal or payment data, never submit.** If a page asks for personal data before showing the total, record the total shown at that step and mark extras.
4. Record the final total and extras in `/admin/audit`, attach a screenshot.
5. Close tabs. Next provider.

- Spread tests over time (not 50 at once), log the date. Aim for 10–15 per session, 2–3 sessions a week.
- Write a short note in the PR/issue each session: tests done, surprising findings (these become content).

Done when

- [ ] ≥ 150 tests recorded; each core provider has ≥ 10 tests

C-034 · Provider pages and methodology with live data

Owner: CLOUD · Size: S · Depends on: C-032 · Status: TODO · PR: – `/anbieter/[slug]`: score, tests, match share, average markup, common extras, last test date. `/methodik` gets live numbers. Neutral, factual wording (CLAUDE.md rule 8).

Phase 4: Launch

L-040 · Real-device QA

Owner: LOCAL · Size: M · Depends on: C-031, C-034 · Status: TODO · PR: – Run through the full journey on Zaid's PC (Chrome, Edge) and his iPhone 15 (Safari, via the staging URL): search, stay page, watch signup, confirmation email, alert email (forced on staging), unsubscribe, legal pages, EN version. Write a fix list as new tasks; fix the small ones directly.

C-041 · Launch hardening

Owner: CLOUD · Size: M · Depends on: C-031 · Status: TODO · PR: – Lighthouse ≥ 90 (mobile) on home, stay, hotel; axe accessibility checks in Playwright; error monitoring (Workers Logs + a daily error digest email to Zaid, or Sentry free if the connector is set up); Supabase backups confirmed (daily on free plan; document restore); 404/500 pages; security headers (CSP, HSTS, Referrer-Policy); rate limits on API routes.

C-042 · Trust report page

Owner: CLOUD · Size: S · Depends on: L-033, C-034 · Status: TODO · PR: – `/report/checkout-test-2026`: "Wir haben {n} Checkouts getestet": charts of match rate per provider, average markup, most common extras, methodology link, dated. Shareable OG image. This is the launch story.

C-043 · Launch content pack

Owner: CLOUD · Size: S · Depends on: C-042 · Status: TODO · PR: – `docs/content/launch/`: 5 TikTok/Reels scripts (hook in the first 2 seconds, 20–40 s, on-screen text, which screen recording to make), a mydealz post draft, 2 Reddit post drafts (r/de, r/reisen; follow the subreddit rules, no spam), images exported from the site. In German.

C-044 · Go live

Owner: CLOUD · Size: S · Depends on: C-041, L-040, Z-12 · Status: TODO · PR: – Final launch checklist (below), production deploy to tidaq.com, collector prod caps set, Search Console sitemap submitted, smoke test in production, announce in the weekly summary issue.

Phase 5: Growth (summaries; expand into full specs when started)

- **C-050 Affiliate wiring:** per approved program, fill `providers.affiliate_template`, test the redirect, add the program to `/affiliate-hinweis`.
- **C-051 Weekly content pack:** a Claude routine (or the first session each Sunday) writes 3 short-video scripts and 1 post from the week's data into `docs/content/YYYY-Www/`.
- **C-052 SEO pages:** "Beste Zeit, [Hotel] zu buchen" (needs 30+ days of data), "Silvester in [Stadt]".
- **C-053 Coverage expansion:** more hotels as budget allows; a "Hotel vorschlagen" form; track user-requested hotels.
- **C-054 Newsletter:** weekly deals email for opt-in subscribers.
- **C-055 Breakfast and room filters:** only if C-010 shows reliable data.
- **C-056 Watch my booking (v2):** forward a booking confirmation → we watch the same stay → alert when it's cheaper to rebook (refundable bookings). Research and spec only.

Checklists

Definition of done (every task)

- [ ] Acceptance criteria of the task all true
- [ ] Tests added/updated; `pnpm lint && pnpm typecheck && pnpm test && pnpm build` green locally and in CI
- [ ] UI tasks: checked at 390 px and 1280 px, DE and EN, keyboard, empty and error states

- [ ] No secrets, no personal data in logs, no new cookies
- [ ] Docs updated (TASKS status, README/PLAN if behavior changed)
- [ ] Self-review of the full diff done (CLAUDE.md §5)

Launch checklist (C-044)

- [ ] Collector ran successfully 7 days in a row; cost on track
- [ ] ≥ 36 hotels with ≥ 14 days of data
- [ ] ≥ 150 checkout tests; provider pages show scores
- [ ] Watch + alert + unsubscribe tested in production with Zaid's email
- [ ] Legal pages final, Zaid read them (Z-12), Steuernummer filled or line removed
- [ ] Lighthouse mobile ≥ 90 on home, stay, hotel; no console errors
- [ ] Sitemap submitted; staging noindex; prod indexable
- [ ] Error alerts reach Zaid's email
- [ ] Launch content pack ready

Weekly review (Zaid, 15 min, Sundays)

- [ ] Read the "Weekly summary" issue
- [ ] Do the Zaid steps listed there (or reply "later" with a date)
- [ ] Check BLOCKERS.md is empty or answered
- [ ] Glance at /admin: data healthy, cost on track

PART D

Design system

Soft Modern: tokens, components, layouts, copy.

Chosen by Zaid from six full-page directions (D-20). Visual reference: `docs/design/reference-homepage.html` (open it in a browser; it is the Soft Modern homepage with example data). Build to this spec; when the reference and this file disagree, this file wins.

1. Feel

Friendly, calm, premium-but-approachable, like a good fintech app for travel. Rounded shapes, soft lavender light, generous white cards, one confident violet accent. Numbers are big and clear. Never "deal site" loud: no red banners, no countdowns, no stock-photo clutter.

Three words: **clear, warm, trustworthy.**

Borrowed from Riviera Editorial (Zaid liked it too), used sparingly: photo-forward deal cards with a soft sky gradient placeholder, and an occasional italic accent word in hero headlines.

2. Tokens

Color (light, the default)

Token	Hex	Use
`--bg`	`#F5F3FC`	page background (lavender-white)
`--surface`	`#FFFFFF`	cards, panels, search card
`--field`	`#F8F6FF`	input backgrounds
`--line`	`#E6E0F7`	borders, dividers, chart grid
`--fg`	`#221C3B`	text, headlines
`--muted`	`#6C6687`	secondary text, labels, axis text
`--accent`	`#6A49E0`	primary buttons, links, focus ring, active chips
`--accent-ink`	`#FFFFFF`	text on accent
`--accent-soft`	`#EDE7FF`	selected chip background, hover
`--good`	`#178A62`	falling price, near-low state, high trust
`--bad`	`#D1435B`	rising price, low trust, checkout markup
`--warn`	`#B26A00`	"collecting data", caution notes
`--blob-1`	`#DDD2FF`	decorative background blob (top left)
`--blob-2`	`#FFD6E5`	decorative background blob (right)

Color (dark, phase 2)

Provide tokens now, ship later: `--bg #14111F`, `--surface #1D1930`, `--field #221D38`, `--line #2E2848`, `--fg #EEEAFB`, `--muted #A39DC0`, `--accent #9C85FF`, `--accent-ink #14111F`, `--good #4FD1A0`, `--bad #FF7A90`. Respect `prefers-color-scheme` once enabled.

Contrast: `--muted` on `--bg` ≈ 5:1 (AA). Never put `--muted` text on `--accent-soft` below 14 px.

Typography

- Family: **Plus Jakarta Sans** (Google Fonts via `next/font`), weights 500, 700, 800. Fallback `system-ui, -apple-system, Segoe UI, sans-serif`.
- Numbers: same family with `font-variant-numeric: tabular-nums` everywhere digits align.

Role	Size (mobile → desktop)	Weight	Letter-spacing	Line-height
Display (hero H1)	36 → 60 px	800	-0.03em	1.05
H2 (section)	26 → 40 px	800	-0.03em	1.1
H3 (card title)	18 → 20 px	800	-0.02em	1.2
Price XL (stay page)	38 → 44 px	800	-0.02em	1
Body	15 → 16.5 px	500	0	1.55
Small / meta	13 px	500	0	1.45
Label / eyebrow	12 px, uppercase	700	0.12em	1.3

Headlines use `text-wrap: balance`. Body text max 65 characters per line.

Shape, depth, spacing

- Radius: cards and panels **18 px**; search card **26 px**; inputs **18 px**; chips and buttons **999 px (pill)**; small badges 10 px.
- Shadow (cards that float): `0 14px 36px rgba(106, 73, 224, 0.10)`. Flat cards use only a 1 px `--line` border. Don't put shadows on everything; only the search card, the stay panel and hover states.
- Spacing scale (px): 4, 8, 12, 16, 20, 24, 32, 40, 56, 72. Section padding 64 px desktop, 44 px mobile. Page gutter 28 px desktop, 18 px mobile. Max content width 1080 px.
- Background blobs: two large blurred radial gradients (`--blob-1` top-left, `--blob-2` right), only on home and landing pages, behind content, `pointer-events: none`.

Motion

- Durations 150–250 ms, easing `cubic-bezier(.2,.8,.2,1)`.
- Allowed: hover lift on cards (translateY −2 px + shadow), chart line draw-in once on first view (600 ms), number count-up on the price (only on first load), sheet/popover fade+scale.
- Respect `prefers-reduced-motion`: no draw-in, no count-up.

3. Components

Component	Spec
Logo	Wordmark `tidaq` lowercase, Plus Jakarta Sans 800, −0.04em. The tail of the `q` may become a small downward wave (price falling); provide SVG + favicon (rounded square, violet bg, white `t`).
Button	Pill. Primary: `--accent` bg, white text, 600 weight, 14–15 px, padding 10×18 (lg 13×22). Ghost: transparent, 1 px `--line`, `--fg` text. Focus: 2 px `--accent` ring, 2 px offset. Disabled: 50 % opacity, no shadow.
Search card	White, radius 26, shadow, padding 18. Fields stacked: "Hotel oder Stadt", then a 2-col row "Daten" + "Gäste", then toggle chips, then a full-width primary button "Preis verfolgen".
Field	Label on top (11 px uppercase `--muted`), value 15 px 700, `--field` bg, 1 px `--line`, radius 18, padding 10×12. Error: `--bad` border + message under it.
Chip / Toggle	Pill, 1 px `--line`, 13 px. On: `--accent` border, `--accent-soft` bg, `--fg` text.
StateChip	Pill, 12 px 700. near_low "Nahe Tiefpunkt" and good "Guter Preis": `--good` text on 16 % `--good` bg. normal "Normal": `--muted` on `--line`. high "Gerade teuer": `--bad` text on 14 % `--bad` bg. collecting "Daten werden gesammelt (3/7)": `--warn`.
PriceDelta	"▼ 5 % unter dem 30-Tage-Schnitt" in `--good`, "▲ 4 % über …" in `--bad`. Tabular numbers.
TrustScore	Small rounded box (radius 10) with the number (13 px 800): ≥ 85 `--good` on 18 % `--good`; 60–84 `--muted` on `--line`; < 60 `--bad` on 18 % `--bad`. Under 5 tests: text "Noch zu wenig Tests" instead of a number. Always with the sample line "24 von 25 Tests stimmten" nearby.
Provider row	Grid: name + note (13 px muted) · price (right-aligned, 15 px 800, tabular) · TrustScore · "Zum Angebot" ghost button. Expected checkout price shown under the price in `--bad` small text when markup is known.
Price chart	SVG, height ~240 desktop / 200 mobile. Line 2.2 px `--good` (or `--accent` when the trend is rising), area fill 10 % opacity, grid lines `--line` 1 px, y labels every 50 € in `--muted` 11 px, x labels 4–5 dates, dashed `--muted` line for the 30-day average with a label, low point as a hollow circle with label "Tief 1.162 €", today as a filled dot. Tooltip on hover/tap: date + price + provider. Data table alternative for screen readers.
Stats row	Three bordered boxes: Tief / Schnitt / Hoch, label 11 px uppercase muted, value 17 px 800 tabular.
Advice box	`--good` 10 % bg, 1 px `--good` 30 % border, radius 18, padding 12×14, bold first sentence.
Deal card	Photo (or sky gradient placeholder `linear-gradient(135deg, #D9CFFC, #FFD9E6)`), radius 18, title 18 px 800, meta 13 px muted, price + "▼ 14 %" in `--good`. Hover lift.
Calendar cell	44×44 px min, radius 12, background tinted by price level (5 steps from `--good` 20 % to `--bad` 20 %), price text inside (11 px 700). Today outlined.
Empty / collecting state	Illustration-free: icon + short headline + one sentence + action. Example: "Wir sammeln noch Daten (3/7 Tage). Der Verlauf wird jeden Tag genauer." + "Preis beobachten".

4. Page layouts

- **Home:** blobs → navbar → centered hero (eyebrow, H1, sub, proof row, search card centered at max 560 px) → live example panel (two columns desktop: chart left, providers right) → how it works (3 cards) → trust figures → deals (3 cards) → CTA → footer.

- **Stay page:** compact header (hotel, dates, guests, edit button) → panel (price + state + delta, chart, stats, advice) → providers table → watch CTA card → "Ähnliche Termine" (cheaper dates in the same hotel) → methodology note.
- **Hotel page:** hero with photo/placeholder and facts → price calendar → best upcoming stays → providers summary.
- **Mobile first:** everything single-column under 720 px; sticky bottom bar on the stay page with price + "Preis beobachten".

---

5. Copy style

- German, informal "du", short. Numbers over adjectives: "9 % günstiger als sonst" instead of "super Deal".
- Buttons say what happens: "Preis verfolgen", "Preis beobachten", "Zum Angebot".
- Never promise the future; never shame providers (facts only).
- Price format: `1.184 €` (DE), `€1,184` (EN). Dates: `1.–3. Dez. 2026` (DE), `1–3 Dec 2026` (EN).

PART E

Decisions

Locked. Sessions follow these without asking.

Locked decisions for Tidaq. Sessions follow these without asking. To change one, add a new entry at the bottom with date, the change and the reason. Never edit an old entry silently.

Last updated: 11 Oct 2026 (kickoff with Zaid).

Product

#	Decision	Why
D-01	**Name: Tidaq** (tide + Nasdaq: "the stock exchange for hotel prices"). Domain **tidaq.com**. Wordmark lowercase `tidaq`.	Short, free .com, matches the price-chart idea and Zaid's investing background.
D-02	**Product:** price history, low/high signal and checkout-trust score for a specific luxury hotel stay (hotel + dates + guests + options).	The gap: Google Hotels shows little history, no breakfast/room filters, and its cheapest providers often cost more at checkout.
D-03	**Segment: five-star and luxury hotels only** in v1.	A 5–10 % move is €100+ per stay. Buyers care, and our commission per booking is high.
D-04	**Coverage at launch: DACH + top EU cities:** Berlin, München, Hamburg, Frankfurt, Wien, Zürich, Paris, London, Mailand, Rom, Barcelona, Amsterdam.	Where German-speaking luxury travellers actually go.
D-05	**Language: German first, English second.** German at `/`, English at `/en`. Informal "du".	The German market is the focus; English adds reach at almost no extra cost when built in from day one.
D-06	**Monetization v1: free for users + affiliate commission** on outbound booking links. No paywall, no subscription in v1.	Lowest barrier while we validate demand. Pro tier or "watch my booking" success fee may come later.
D-07	**Target user:** "aspirational luxury traveller", 25–45, books a five-star hotel 1–3× a year, price-aware, research-heavy, DACH.	Not the ultra-rich (they book through agents and loyalty programs).
D-08	**Alerts: email** (double opt-in). Telegram and web push later.	Email is universal and legally clear.

Tech

#	Decision	Why
D-10	**Hosting: Cloudflare.** Web app = Next.js (App Router) on Cloudflare Workers via `@opennextjs/cloudflare`. Data collector = separate Cloudflare Worker with Cron Triggers. **No Vercel.**	Zaid's choice. Cheap, fast, EU-friendly, cron built in.
D-11	**Database: Supabase Postgres, region Frankfurt (eu-central-1).** Supabase Auth for magic-link emails if needed.	Zaid already knows Supabase (AheadSignal). Postgres is right for time series and stats. EU data residency.

#	Decision	Why
D-12	**Price data: DataForSEO Google Hotels API** (Hotel Info + Hotel Searches, standard queue). About $0.0008 per hotel entity per task (Oct 2026 pricing page). SerpApi is the fallback.	Licensed data access instead of scraping provider sites ourselves. Cheap enough to track ~50 hotels daily.
D-13	**Budget: under €50/month** for data + hosting in the first 3 months. Hard daily cap in code (`DAILY_TASK_CAP`).	Zaid's budget.
D-14	**Email: Resend** (free tier, 3,000 emails/month), domain `tidaq.com`. Templates with React Email.	Simple API, good deliverability, works from Workers.
D-15	**Analytics: Cloudflare Web Analytics (cookieless)** + our own event table. **No tracking cookies, so no cookie banner.**	DSGVO-simple and good UX.
D-16	**Language/tooling:** TypeScript strict, pnpm, Tailwind CSS, next-intl, Vitest, Playwright, ESLint + Prettier, Zod for all external data.	Mainstream, AI-friendly, well-tested.
D-17	**Charts: own SVG components** (d3-scale/d3-shape only for math). No heavy chart library.	Full design control, tiny bundle, matches the reference design.
D-18	**Repo: `zaidg7/tidaq`, private.** Default branch `main`.	Keeps data pipeline and trust method private.

Design

#	Decision	Why
D-20	**Design direction: Soft Modern.** Rounded, friendly, app-like (lavender/violet accent, Plus Jakarta Sans). Zaid also liked Riviera Editorial; borrow its photo-forward deal cards and sky gradient as accents only. Spec in `docs/DESIGN.md`, reference in `docs/design/reference-homepage.html`.	Zaid's pick from six full-page mockups.
D-21	**Prices in German format:** `1.184 €` (amount, space, €; no cents for totals). English: `€1,184`. Always use `Intl.NumberFormat`.	Correct German typography. The mockup showed `€1.184`, which is wrong for DE.
D-22	**Falling prices are green, rising are red.**	Falling is good for the user.

Operating model

#	Decision	Why
D-30	**Two AI sessions do the work:** CLOUD (claude.ai cloud session, main builder) and LOCAL (Claude Code on Zaid's Windows 11 PC, for tasks needing his browser, accounts or a real machine). Zaid does only ZAID tasks.	Zaid has ~5–7 h/week (school until mid-2027, other businesses, gym).
D-31	**Auto-merge: sessions squash-merge their own PRs when CI is green** and the PR checklist is done. Zaid does not review code.	Zaid's choice. Free private repos have no branch protection, so the session merges itself after verifying green CI.
D-32	**Claude plan: Pro** (may upgrade to Max 5×). On Pro, run **one session at a time**; CLOUD is the default. LOCAL only for LOCAL tasks.	Pro usage limits.

#	Decision	Why
D-33	**Progress updates: weekly summary** every Sunday as a GitHub issue assigned to `zaidg7` plus `docs/progress/YYYY-Www.md`. Blockers go to `docs/BLOCKERS.md` immediately.	Zaid gets a phone notification once a week.
D-34	**Baseline hotels: Claude picks** (seed list in `docs/TASKS.md`, C-011).	Zaid's choice.
D-35	**Impressum/Datenschutz: reuse AheadSignal details** (Zaid Ghazal, Kleinunternehmer § 19 UStG), cite **§ 5 DDG** (not TMG; TMG was replaced on 14 May 2024).	Same business owner.
D-36	**Domain registrar: Cloudflare Registrar** (at-cost). `.com` only for now.	Zaid's choice. He doesn't need .de.

PART F

Operating manual for Claude sessions

CLAUDE.md: what every session reads first.

You are one of the engineers building **Tidaq**, a price tracker for luxury hotel stays. The owner is **Zaid Ghazal** (GitHub `zaidg7`). He is 18, still in school, and has about 5–7 hours a week. He does not review code. **You and the other session are the engineering team.** Work like a senior engineer who owns the outcome: careful, tested, small steps, no guessing.

Read these before doing anything else, in this order:

1. This file
2. `docs/DECISIONS.md`: locked decisions, don't re-ask
3. `docs/TASKS.md`: the backlog, your work queue
4. `docs/BLOCKERS.md`: open questions for Zaid
5. `docs/PLAN.md`: product, architecture, data model, methods (read the parts your task touches)
6. `docs/DESIGN.md` and `docs/design/reference-homepage.html`: before any UI work

1. Which session are you?

- **CLOUD**: a claude.ai cloud session (Linux container). The main builder. You take tasks with owner `CLOUD`.
- **LOCAL**: Claude Code on Zaid's Windows 11 PC. You take tasks with owner `LOCAL` first: anything that needs a real browser (Claude in Chrome), Zaid's logged-in accounts, or a physical machine. If no LOCAL task is open, you may take a `CLOUD` task marked `parallel-safe`.

If you're unsure: a Linux container under `/home/user` means CLOUD; a Windows path means LOCAL.

2. Session protocol

Start of session

1. `git fetch origin && git checkout main && git pull`.
2. Read the files listed above.
3. List open PRs (GitHub MCP tools). **An open PR is a lock:** a task with an open PR titled `[ID] …` is taken.
4. Pick the **first** task in the overview table of `docs/TASKS.md` (it is in priority order) that is `TODO`, has your owner, has all `Depends on` tasks `DONE`, and has no open PR. Zaid tasks (`Z-xx`) count as done when the "Zaid tasks" table in TASKS.md says so, or when you can see the result (e.g. the secret exists because a workflow using it succeeds). If unsure, try; if it fails for missing access, set the task `BLOCKED` and add a BLOCKERS entry.
5. Create the branch `task/<ID>-<short-slug>` (e.g. `task/C-013-collector-worker`) and open a **draft PR** titled `[<ID>] <task title>` right away. That claims the task.

While working

- One task = one PR. Aim for under ~500 changed lines. If a task is bigger, split it into `C-013a`, `C-013b` in `TASKS.md` within the same PR and finish part (a) first.

- Write tests with the code (see §5). Run the checks locally before every push: `pnpm lint && pnpm typecheck && pnpm test && pnpm build`.
- Commit small and often with clear messages (`feat(collector): …`, `fix(web): …`, `test: …`, `docs: …`).
- If you discover work outside your task, add it to `docs/TASKS.md` as a new `TODO` task in your PR. Don't do it now.

Finishing a task

1. Update the task in `docs/TASKS.md`: status `DONE`, PR link, one-line note of anything the next session must know.
2. Fill in the PR template checklist honestly.
3. Mark the PR ready for review. Wait for CI. **If CI is green on the latest commit and every checklist box is true, squash-merge the PR yourself** and delete the branch (decision D-31). If CI is red: fix it. Never merge red. Never skip, disable or weaken a test to get green.
4. After the merge, check the `main` deploy job succeeded. If it failed, fixing it is your next task.
5. Pick the next task. Keep going until the usage limit is near or only blocked tasks are left.

End of session

- If you stop mid-task: push your branch and leave a PR comment with **Status / Next step / Open questions**, so the next session can continue.
- If you're blocked by something only Zaid can do: add it to `docs/BLOCKERS.md` (format inside), set the task to `BLOCKED`, and move to another task. Blocker-only edits may be committed straight to `main`.

Weekly summary (Sundays, or the first session after)

Create `docs/progress/YYYY-Www.md` from `docs/progress/TEMPLATE.md` and open a GitHub **issue** titled `Weekly summary - Week NN` assigned to `zaidg7`, with the same text. Keep it short and in plain English: what shipped (with links), what's next, what Zaid needs to do (exact steps, time estimate), data cost this week.

---

3. Hard rules (never break)

1. **Never commit secrets.** API keys live in GitHub Actions secrets, Cloudflare Worker secrets and the Claude cloud environment. Read them from `process.env` / Worker `env`. `.env*` files are gitignored. If you find a secret in the code, remove it, rotate it via BLOCKERS and tell Zaid.
2. **Never scrape provider websites from our servers** (Booking, Expedia, hotel sites). Price data comes from the DataForSEO API only. Checkout tests are manual (task L-033).
3. **Never make a booking, enter payment data or personal data** on any booking site, including in LOCAL browser sessions. Stop at the last page before payment.
4. **Never exceed the data budget.** Every DataForSEO call goes through the budget guard (`DAILY_TASK_CAP`, monthly cap). No loops that call paid APIs without the guard.
5. **Never send email to real users from dev or preview.** Dev uses Resend's test mode or the `ALERTS_DRY_RUN=true` flag.
6. **Never delete production data** or run destructive SQL on production. Schema changes only via migrations in `supabase/migrations/`.
7. **Never change a decision in `docs/DECISIONS.md` silently.** If a decision blocks you, propose the change in BLOCKERS and keep working on something else.

8. **Legal text:** never call a provider a liar or a scam. Show facts: "In 6 of 25 tests the checkout price was higher." Provider claims need data with date and screenshot (see PLAN §8).
9. **No dark patterns:** no fake urgency, no fake scarcity, no pre-ticked boxes, no hidden affiliate links. Affiliate links are labeled and use `rel="sponsored nofollow"`.
10. **Accessibility and performance are part of done:** keyboard usable, visible focus, contrast AA, Lighthouse performance ≥ 90 on mobile for key pages.

4. Repository map (target)

```
apps/web/            Next.js app (App Router), deployed to Cloudflare Workers via OpenNext
  app/[locale]/…     pages (de default, en)
  components/        UI components (Soft Modern design system)
  lib/               server utils, data access, formatting, stats
  messages/          next-intl de.json / en.json
apps/collector/      Cloudflare Worker: cron → DataForSEO → Supabase
packages/core/       shared TypeScript: types, zod schemas, price stats, trust score
supabase/migrations/ SQL migrations (source of truth for the schema)
tests/fixtures/      recorded DataForSEO responses (no personal data)
docs/                plan, tasks, decisions, design, research, progress
.github/workflows/   CI and deploy
```

pnpm workspace. Node 22 LTS. If reality differs, update this map in your PR.

5. Quality bar

- **TypeScript strict**, no `any` without a comment explaining why. Validate every external response with **Zod** before it touches the database.
- **Tests:** unit tests (Vitest) for all logic in `packages/core` (stats, trust score, normalization, budget guard) with ≥ 90 % line coverage there; integration tests for the collector using recorded fixtures (never live API in CI); Playwright smoke tests for key pages (home, stay page, legal pages) in both locales.
- **UI:** follow `docs/DESIGN.md` exactly. Compare against `docs/design/reference-homepage.html`. Test at 390 px and 1280 px. Real German copy, no lorem ipsum.
- **Copy:** German first, informal "du", short sentences, no hype words ("revolutionär", "unschlagbar"). Every user-facing string goes through next-intl. Provide `en.json` at the same time.
- **Money:** store integer cents in EUR. Format only at the edge with `Intl.NumberFormat` (`1.184 €` in DE).
- **Dates:** store UTC timestamps; stay dates as `date` (no time). Hotel nights are check-in to check-out.
- **Errors:** user-facing errors explain what happened and what to do. Log server errors with context; never log emails or personal data.
- **Self-review before merge:** re-read the full diff as a strict reviewer. Ask: what breaks in production? What happens with empty data, one data point, API errors, a timezone edge, German umlauts, mobile width?

6. How to work with Zaid

- He doesn't read code. Talk in outcomes: "Stay pages now show the 60-day chart" instead of "Added StayChart component".
- Ask him only for things only he can do (accounts, payments, identity, his browser, decisions). Put them in BLOCKERS with **exact steps** and how long it takes ("2 min: open X, click Y, paste into Z").

- **Never ask him to paste a key, token or password into a chat, issue or PR.** Tell him where to store it (GitHub secret name, Cloudflare secret name, Claude environment variable name).
- He prefers direct, no-fluff English. No "Great question!".

7. Secrets and environment variables

Name	Where it lives	Used by
`DATAFORSEO_LOGIN`, `DATAFORSEO_PASSWORD`	GitHub secret, Cloudflare secret (collector), Claude cloud env	collector, spike scripts
`SUPABASE_URL`, `SUPABASE_ANON_KEY`	GitHub secret, Cloudflare var (web)	web
`SUPABASE_SERVICE_ROLE_KEY`	GitHub secret, Cloudflare secret (collector + web server only)	server-side writes
`RESEND_API_KEY`	GitHub secret, Cloudflare secret	alerts
`CLOUDFLARE_API_TOKEN`, `CLOUDFLARE_ACCOUNT_ID`	GitHub secret	deploy workflow
`ADMIN_EMAILS`	Cloudflare var	admin pages (`zaidghazal216@gmail.com`)
`DAILY_TASK_CAP` (default 600), `MONTHLY_BUDGET_USD` (default 30)	Cloudflare var	budget guard
`ALERTS_DRY_RUN`	Cloudflare var (`true` outside production)	alert engine

CI injects secrets into deploys. Sessions do not need raw keys for normal work: run code against fixtures, deploy through CI, and inspect production data through the Supabase MCP tools when available.

8. Useful facts

- Today's plan start: Mon 12 Oct 2026. Soft launch target: **Mon 30 Nov 2026**.
- DataForSEO Hotel Info: `check_in` / `check_out` (max 30 nights), `adults` (default 2), `currency` (use `EUR`), `language_code` `de`, location Germany. Billed per hotel entity, around $0.0008 in the standard queue. Results can be fetched free for 30 days. **Verify the real response shape in C-010 before building on it.**
- Cloudflare Workers paid plan ($5/month) is expected: the Next.js bundle likely exceeds the free 3 MB limit. It's included in the €50 budget plan; Zaid activates it in Z-04 if the free plan fails (try free first).
- Supabase free projects pause after a week without activity; the daily collector keeps the project active.